

Bloque 7 — Mini-especificaciones funcionales y técnicas

Rendiciones: R010 a R017

1. Propósito del bloque

Este bloque implementa el proceso completo que comienza cuando el responsable termina de registrar sus gastos y decide presentar la rendición.

Incluye:

- Presentación formal.
- Bloqueo de edición.
- Asignación e inicio de revisión.
- Revisión individual de cada gasto.
- Observaciones y correcciones.
- Aprobación total, parcial o rechazo.
- Historial completo de decisiones.
- Liquidación financiera.
- Devoluciones del responsable.
- Reembolsos de la empresa.
- Conciliación con movimientos bancarios.
- Cierre definitivo de la rendición.

El flujo general será:

```
BORRADOR
→ PRESENTADA
→ EN_REVISIÓN
  └─ OBSERVADA → PRESENTADA nuevamente
  └─ RECHAZADA
  └─ APROBADA
      → LIQUIDADA
      → CERRADA
```

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
R010	Presentación de rendición	4 h	Backend	Parcial	Bloqueo y registro formal
R011	Revisión y observaciones	6 h	Frontend	Parcial	Revisión por gasto y corrección
R012	Aprobación y rechazo	6 h	Backend	Parcial	Decisión controlada por rol
R013	Historial de aprobaciones	4 h	Frontend	Alta, con ampliaciones	Línea de tiempo completa
R014	Liquidación de rendición	8 h	Backend	Alta	Resultado económico congelado
R015	Devolución del responsable	8 h	Backend	Parcial	Registro de ingresos por devolución
R016	Reembolso al responsable	6 h	Backend	Parcial	Orden y ejecución del pago
R017	Conciliación de liquidaciones	10 h	Integraciones	Alta, con ajustes	Vínculo con movimientos bancarios

El bloque suma **52 horas**.

La Carta Gantt exige:

- Bloqueo de edición y registro de presentación.
- Observaciones por gasto y devolución a corrección.
- Aprobación por rol, rechazo motivado y trazabilidad.
- Secuencia completa visible al usuario autorizado.
- Liquidación cuadrada, con devolución o con reembolso.
- Devolución enlazada con el ingreso bancario.
- Reembolso con orden de pago y egreso bancario.
- Aplicación bancaria a devoluciones y reembolsos.

3. Separación de conceptos

Es fundamental no utilizar un único campo para representar todo el proceso.

3.1 Estado de la rendición

Representa la etapa global:

BORRADOR
PRESENTADA
EN_REVISION
OBSERVADA
APROBADA
RECHAZADA
LIQUIDADA
CERRADA
ANULADA

Estos estados ya están incluidos en `Rendicion`.

3.2 Estado de revisión del gasto

Representa la decisión sobre cada detalle:

PENDIENTE
OBSERVADO
APROBADO
RECHAZADO

Un gasto puede aprobarse, observarse o rechazarse independientemente de los demás.

3.3 Resultado de liquidación

Representa el resultado financiero:

CUADRADA
RESPONSABLE_DEVUELVE
EMPRESA_REEMBOLSA

`LiquidacionRendicion` ya contiene estos resultados y conserva un snapshot de fondos, gastos aprobados y diferencia.

3.4 Estado de regularización

Representa si la diferencia ya fue efectivamente pagada y conciliada:

PENDIENTE
PARCIAL
REGULARIZADA
CONCILIADA
ANULADA

Este estado no existe actualmente y deberá incorporarse.

4. Flujo completo de estados

Flujo normal

BORRADOR
→ PRESENTADA
→ EN_REVISION
→ APROBADA
→ LIQUIDADA
→ CERRADA

Flujo con observaciones

BORRADOR
→ PRESENTADA
→ EN_REVISION
→ OBSERVADA
→ corrección del responsable
→ PRESENTADA
→ EN_REVISION
→ APROBADA

Flujo rechazado

PRESENTADA
→ EN_REVISION
→ RECHAZADA

Flujo extraordinario

APROBADA, LIQUIDADA o CERRADA
→ REABIERTA

La reapertura debe ser excepcional, exigir permiso especial y motivo obligatorio. Su implementación completa se reforzará en B016.

5. Cobertura actual del modelo

`AprobacionRendicion` ya registra:

- Rendición.
- Acción.
- Fecha.
- Comentario.
- Usuario.

Las acciones disponibles incluyen presentación, inicio de revisión, observación, aprobación, rechazo y reapertura.

`LiquidacionRendicion` ya:

- Mantiene una relación uno a uno con la rendición.
- Guarda fondos entregados.
- Guarda gastos aprobados.
- Calcula diferencia.
- Determina el resultado.
- Calcula monto regularizado.
- Calcula saldo pendiente.

También valida que:

$$\text{diferencia} = \text{fondos entregados} - \text{gastos aprobados}$$

y que el resultado sea consistente con el signo de la diferencia.

`PagoLiquidacionRendicion` ya admite:

- Devolución del responsable.
- Reembolso de la empresa.
- Transferencia, efectivo, débito u otro medio.
- Pagos parciales.
- Anulación.

Además, evita regularizar un monto superior a la diferencia de la liquidación.

6. Ajustes principales requeridos

6.1 Historial estructurado de revisión de gastos

Actualmente `DetalleRendicion` solo conserva:

```
estado_revision  
monto_aprobado  
motivo_observacion
```

Si el mismo gasto se observa dos veces, el motivo anterior puede sobrescribirse.

Se propone crear:

```
class RevisionDetalleRendicion(ModeloAuditoria):  
    class Accion(models.TextChoices):  
        OBSERVADO = "OBSERVADO", "Observado"  
        CORREGIDO = "CORREGIDO", "Corregido"  
        APROBADO = "APROBADO", "Aprobado"  
        RECHAZADO = "RECHAZADO", "Rechazado"  
        REABIERTO = "REABIERTO", "Reabierto"  
  
    detalle = models.ForeignKey(  
        DetalleRendicion,  
        on_delete=models.PROTECT,  
        related_name="revisiones",  
    )  
    accion = models.CharField(  
        max_length=20,  
        choices=Accion.choices,  
    )  
    estado_anterior = models.CharField(max_length=20)  
    estado_nuevo = models.CharField(max_length=20)  
    monto_anterior = models.DecimalField(  
        max_digits=14,  
        decimal_places=2,  
        null=True,  
        blank=True,  
    )  
    monto_nuevo = models.DecimalField(  
        max_digits=14,  
        decimal_places=2,  
        null=True,  
        blank=True,  
    )  
    comentario = models.TextField(blank=True)  
    usuario = models.ForeignKey(  
        settings.AUTH_USER_MODEL,
```

```

        on_delete=models.SET_NULL,
        null=True,
        related_name="revisiones_detalle_rendicion",
    )
    fecha = models.DateTimeField(default=timezone.now)

```

6.2 Ampliación del historial general

Agregar a `AprobacionRendicion`:

```

estado_anterior = models.CharField(
    max_length=20,
    blank=True,
)

estado_nuevo = models.CharField(
    max_length=20,
    blank=True,
)

datos = models.JSONField(
    default=dict,
    blank=True,
)

correlation_id = models.UUIDField(
    default=uuid.uuid4,
    editable=False,
)

```

El campo `datos` podrá conservar:

```

{
    "total_fondos": "500000.00",
    "total_declarado": "480000.00",
    "total_aprobado": "450000.00",
    "errores_documentales": 0,
    "advertencias": 2
}

```

6.3 Estado de liquidación

Agregar a `LiquidacionRendicion`:

```

class Estado(models.TextChoices):
    PENDIENTE = "PENDIENTE", "Pendiente de regularización"
    PARCIAL = "PARCIAL", "Parcialmente regularizada"
    REGULARIZADA = "REGULARIZADA", "Regularizada"
    CONCILIADA = "CONCILIADA", "Conciliada"
    ANULADA = "ANULADA", "Anulada"

estado = models.CharField(
    max_length=20,
    choices=Estado.choices,
    default=Estado.PENDIENTE,
)

```

6.4 Ciclo de vida del pago de liquidación

El nombre `PagoLiquidacionRendicion` se mantiene para evitar cambios innecesarios, pero debe distinguir:

- Orden o instrucción.
- Ejecución efectiva.
- Conciliación bancaria.
- Anulación.

Agregar:

```

class Estado(models.TextChoices):
    PENDIENTE = "PENDIENTE", "Pendiente"
    EJECUTADO = "EJECUTADO", "Ejecutado"
    PARCIALMENTE_CONCILIADO = (
        "PARCIALMENTE_CONCILIADO",
        "Parcialmente conciliado",
    )
    CONCILIADO = "CONCILIADO", "Conciliado"
    ANULADO = "ANULADO", "Anulado"

estado = models.CharField(
    max_length=30,
    choices=Estado.choices,
    default=Estado.PENDIENTE,
)

fecha_ejecucion = models.DateField(
    null=True,
    blank=True,
)

ejecutado_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,

```



```
on_delete=models.SET_NULL,  
null=True,  
blank=True,  
related_name="pagos_liquidacion_ejecutados",  
)  
  
requiere_conciliacion_bancaria = models.BooleanField(  
    default=True,  
)  
  
motivo_anulacion = models.TextField(blank=True)
```

R010 — Presentación de rendición

1. Objetivo

Permitir que el responsable declare formalmente que terminó de registrar la rendición y la envíe a revisión.

La presentación debe:

- Validar la rendición completa.
- Registrar la acción.
- Cambiar el estado.
- Bloquear la edición ordinaria.
- Dejar disponible la rendición para revisión.

2. Estado permitido

Solo se puede presentar desde:

BORRADOR
OBSERVADA

Resultado:

PRESENTADA

3. Validaciones previas obligatorias

Antes de presentar:

1. El responsable está definido.
2. El período es válido.
3. Existe motivo o descripción del cometido.
4. Existe al menos un gasto.
5. Todos los gastos tienen categoría.
6. Categoría y subcategoría son coherentes.
7. No existen errores documentales abiertos.
8. Todo gasto sin documento tiene justificación.
9. Los adjuntos obligatorios están cargados.
10. Los montos son positivos y consistentes.
11. No existen documentos duplicados bloqueantes.
12. Los fondos asociados pertenecen al responsable.
13. Los totales pueden calcularse.
14. No existe otra presentación concurrente.
15. El usuario tiene derecho sobre la rendición.

No será obligatorio que exista una entrega de fondo, porque puede tratarse de gastos pagados directamente por el responsable que posteriormente serán reembolsados.

4. Declaración del responsable

La pantalla debe mostrar una confirmación:

Declaro que los datos y documentos incorporados son correctos
y corresponden a gastos efectuados en el período informado.

Se recomienda registrar:

```
declaracion_aceptada_en = models.DateTimeField(
    null=True,
    blank=True,
)

declaracion_aceptada_por = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.SET_NULL,
    null=True,
    blank=True,
    related_name="rendiciones_declaradas",
)
```

También puede conservarse la versión del texto aceptado en el evento histórico.

5. Servicio

```
services/rendiciones/workflow.py
```

```
@transaction.atomic
def presentar_rendicion(
    *,
    rendicion_id,
    usuario,
    declaracion_aceptada,
    comentario="",
):
    rendicion = (
        Rendicion.objects
        .select_for_update()
        .get(pk=rendicion_id)
    )

    # Validar permiso.
    # Validar estado.
    # Ejecutar validación completa.
    # Comprobar errores bloqueantes.
    # Calcular snapshot.
    # Cambiar a PRESENTADA.
    # Registrar fecha_presentacion.
    # Crear AprobacionRendicion.
    # Crear EventoAuditoria.
```

6. Bloqueo de edición

Después de presentar:

- La cabecera queda de solo lectura.
- Los gastos no pueden agregarse.
- Los gastos no pueden editarse.
- Los adjuntos no pueden reemplazarse.
- Los fondos no pueden vincularse ni modificarse.

Solo podrá revertirse el bloqueo mediante:

```
OBSERVADA
REABIERTA
```

7. URLs

```
/administracion/rendiciones/<pk>/presentar/  
/administracion/rendiciones/<pk>/validar-presentacion/
```

8. Vistas

```
RendicionValidarPresentacionView  
RendicionPresentarView
```

La acción de presentación debe ejecutarse mediante `POST`.

9. Interfaz

Mostrar antes de confirmar:

Indicador	Resultado
Gastos	12
Fondos recibidos	\$500.000 CLP
Total declarado	\$480.000 CLP
Errores	0
Advertencias	2

Las advertencias deben poder abrirse y revisarse.

10. Permisos

```
presentar_rendicion  
presentar_rendicion ajena
```

El responsable no puede presentar una rendición de otra persona salvo delegación o permiso especial.

11. Auditoría

Registrar:

- Estado anterior.

- Estado nuevo.
 - Usuario.
 - Fecha.
 - Declaración.
 - Totales.
 - Número de detalles.
 - Validaciones.
 - Comentario.
-

12. Pruebas

- Presentar borrador válido.
 - Presentar observada corregida.
 - Sin gastos.
 - Con errores documentales.
 - Con advertencias.
 - Sin adjuntos obligatorios.
 - Usuario no responsable.
 - Presentación doble.
 - Bloqueo posterior.
 - Registro histórico.
 - Rollback ante error.
-

13. Criterios de aceptación

- Solo se presenta una rendición válida.
 - Los errores bloquean la acción.
 - Las advertencias permanecen visibles.
 - Se registra la declaración.
 - La rendición queda bloqueada.
 - La acción es transaccional.
 - Existe evidencia histórica.
-

R011 — Revisión y observaciones

1. Objetivo

Permitir que un revisor examine la rendición y decida sobre cada gasto individual.

El revisor podrá:

- Iniciar revisión.
- Visualizar documentos.
- Observar un gasto.
- Aprobarlo total o parcialmente.

- Rechazarlo.
 - Devolver toda la rendición para corrección.
-

2. Inicio de revisión

Estado inicial:

PRESENTADA

Acción:

INICIAR REVISIÓN

Resultado:

EN_REVISION

Se recomienda agregar a **Rendicion** :

```
revisor_actual = models.ForeignKey(  
    settings.AUTH_USER_MODEL,  
    on_delete=models.SET_NULL,  
    null=True,  
    blank=True,  
    related_name="rendiciones_en_revision",  
)  
  
revision_iniciada_en = models.DateTimeField(  
    null=True,  
    blank=True,  
)
```

3. Control de toma de revisión

Cuando un revisor inicia la tarea:

- La rendición queda asignada.
- Otros revisores pueden verla.
- Solo el revisor asignado o un administrador puede modificar decisiones.
- Puede liberarse o reasignarse mediante acción auditada.

Esto evita revisiones contradictorias.

4. Decisiones por gasto

Aprobar completamente

```
estado_revision = APROBADO  
monto_aprobado = total
```

Aprobar parcialmente

```
estado_revision = APROBADO  
monto_aprobado < total
```

Debe exigirse comentario explicando la diferencia.

Observar

```
estado_revision = OBSERVADO  
monto_aprobado = vacío
```

Debe indicar:

- Motivo.
- Corrección solicitada.
- Si puede reemplazar documento.
- Si puede cambiar monto.
- Si puede cambiar categoría.

Rechazar

```
estado_revision = RECHAZADO  
monto_aprobado = 0
```

El motivo es obligatorio.

5. Observación estructurada

Se recomienda ampliar `RevisionDetalleRendicion` con:

```
class TipoObservacion(models.TextChoices):  
    DOCUMENTO_ILEGIBLE = (  
        "DOCUMENTO_ILEGIBLE",  
        "Documento ilegible",  
    )  
    DOCUMENTO_FALTANTE = (
```

```

        "DOCUMENTO_FALTANTE",
        "Documento faltante",
    )
    DUPLICADO = "DUPLICADO", "Posible duplicado"
    MONTO_INCORRECTO = "MONTO_INCORRECTO", "Monto incorrecto"
    FECHA_INCORRECTA = "FECHA_INCORRECTA", "Fecha incorrecta"
    CATEGORIA_INCORRECTA = (
        "CATEGORIA_INCORRECTA",
        "Categoría incorrecta",
    )
    PROVEEDOR_INCONSISTENTE = (
        "PROVEEDOR_INCONSISTENTE",
        "Proveedor inconsistente",
    )
    GASTO_NO_AUTORIZADO = (
        "GASTO_NO_AUTORIZADO",
        "Gasto no autorizado",
    )
    OTRO = "OTRO", "Otro"

```

6. Devolución a corrección

Cuando exista al menos un gasto observado:

EN_REVISION
→ OBSERVADA

La rendición vuelve al responsable.

Solo podrán modificarse:

- Gastos observados.
- Adjuntos relacionados.
- Campos expresamente habilitados.
- Cabecera, cuando exista una observación sobre ella.

Los gastos ya aprobados permanecen bloqueados.

7. Corrección del responsable

Al corregir un detalle:

- No se borra la observación.
- Se registra una nueva revisión con acción **CORREGIDO**.
- Se conservan valores anteriores.
- El gasto vuelve a estado **PENDIENTE**.

- La rendición continúa **OBSERVADA** hasta presentarse de nuevo.
-

8. Pantalla de revisión

Diseño sugerido:

Panel izquierdo:

- Lista de gastos
- Estado
- Alertas

Panel central:

- Datos del gasto
- Montos
- Proveedor

Panel derecho:

- Vista previa del documento
 - Historial
 - Acciones
-

9. Acciones masivas

Se podrá incorporar:

Aprobar todos los gastos válidos

pero el backend deberá validar cada detalle individualmente.

No se debe permitir una aprobación masiva ciega de gastos con errores o advertencias críticas.

10. URLs

```
/administracion/rendiciones/revision/  
/administracion/rendiciones/<pk>/iniciar-revision/  
/administracion/rendiciones/<pk>/revisar/  
/administracion/rendiciones/gastos/<pk>/aprobar/  
/administracion/rendiciones/gastos/<pk>/observar/  
/administracion/rendiciones/gastos/<pk>/rechazar/  
/administracion/rendiciones/<pk>/devolver-correccion/
```

11. Vistas

```
RendicionesPendientesRevisionListView  
RendicionIniciarRevisionView  
RendicionRevisionView  
DetalleRendicionAprobarView  
DetalleRendicionObservarView  
DetalleRendicionRechazarView  
RendicionDevolverCorreccionView
```

12. Servicio

```
services/rendiciones/revision.py
```

Funciones:

```
iniciar_revision()  
aprobar_detalle()  
aprobar_detalle_parcial()  
observar_detalle()  
rechazar_detalle()  
corregir_detalle_observado()  
devolver_rendicion_correccion()  
reasignar_revisor()
```

13. Permisos

```
revisar_rendicion  
observar_detallerendicion  
aprobar_detallerendicion  
rechazar_detallerendicion  
reasignar_revision_rendicion
```

El responsable no puede revisar ni aprobar su propia rendición.

14. Auditoría

Registrar en cada acción:

- Estado anterior.
- Estado nuevo.

- Monto anterior.
 - Monto aprobado.
 - Observación.
 - Usuario.
 - Fecha.
 - Documentos visibles en ese momento.
-

15. Pruebas

- Iniciar revisión.
 - Doble toma de revisión.
 - Aprobar gasto.
 - Aprobar parcialmente.
 - Aprobar más que el total.
 - Observar sin comentario.
 - Rechazar sin motivo.
 - Devolver a corrección.
 - Corregir gasto observado.
 - Impedir modificar gasto aprobado.
 - Responsable intenta revisar.
 - Historial completo.
-

16. Criterios de aceptación

- La rendición puede asignarse a un revisor.
 - Cada gasto recibe una decisión.
 - Las observaciones quedan estructuradas.
 - Las aprobaciones parciales se justifican.
 - Las correcciones no eliminan historial.
 - No existe autoaprobación.
 - Los cambios quedan auditados.
-

R012 — Aprobación y rechazo

1. Objetivo

Tomar la decisión final sobre la rendición completa una vez revisados todos sus gastos.

2. Estados permitidos

La decisión se realiza desde:

EN_REVISION

Resultados:

APROBADA
RECHAZADA
OBSERVADA

3. Requisitos para aprobar

1. No existen errores documentales abiertos.
2. No existen gastos pendientes.
3. No existen gastos observados.
4. Todos los gastos tienen decisión.
5. Los montos aprobados son válidos.
6. Los totales se recalcularon.
7. El usuario tiene permiso.
8. El usuario no es el responsable.
9. La rendición no cambió después de iniciar la revisión.
10. No existe otra aprobación concurrente.

Una rendición puede aprobarse aunque algunos gastos estén rechazados.

En ese caso:

total aprobado
<
total declarado

La diferencia se reflejará en la liquidación.

4. Rechazo total

El rechazo completo debe utilizarse cuando:

- La rendición no corresponde.
- El gasto o cometido es improcedente.
- Existe fraude o duplicidad grave.
- No puede corregirse mediante observaciones.
- La política administrativa ordena cerrar el proceso como rechazado.

El motivo es obligatorio.

5. Diferencia entre observar y rechazar

Observar

La rendición puede corregirse.

Rechazar

La rendición se da por no aceptada.

Una rendición rechazada no vuelve automáticamente a borrador.

La reapertura requiere permiso especial.

6. Servicio

services/rendiciones/aprobacion.py

```
@transaction.atomic
def aprobar_rendicion(
    *,
    rendicion_id,
    usuario,
    comentario="",
):
    rendicion = (
        Rendicion.objects
        .select_for_update()
        .select_related("responsable")
        .get(pk=rendicion_id)
    )

    # Validar permiso.
    # Evitar autoaprobación.
    # Verificar estado.
    # Bloquear detalles.
    # Verificar decisiones.
    # Calcular total aprobado.
    # Cambiar estado.
    # Registrar fecha_aprobacion.
    # Crear evento.
```

Funciones:

```
aprobar_rendicion()  
rechazar_rendicion()  
observar_rendicion()  
validar_decision_final()
```

7. Firma o confirmación

La aprobación deberá requerir una confirmación explícita.

Puede incorporarse:

- Reingreso de contraseña.
- Código interno.
- Firma electrónica.
- Confirmación simple autenticada.

Para el MVP es suficiente una confirmación autenticada, registrada con usuario, fecha e IP.

8. URLs

```
/administracion/rendiciones/<pk>/aprobar/  
/administracion/rendiciones/<pk>/rechazar/  
/administracion/rendiciones/<pk>/observar/
```

9. Vistas

```
RendicionAprobarView  
RendicionRechazarView  
RendicionObservarView
```

10. Permisos

```
aprobar_rendicion  
rechazar_rendicion  
observar_rendicion
```

Puede establecerse una regla adicional por monto:

Hasta \$X CLP → aprobador nivel 1
Sobre \$X CLP → aprobador nivel 2

Esta regla solo debe implementarse si existe una política formal.

11. Auditoría

Registrar:

- Totales.
 - Detalles aprobados.
 - Detalles rechazados.
 - Monto aprobado.
 - Usuario.
 - Fecha.
 - Comentario.
 - Estado anterior y nuevo.
 - Dirección IP.
 - Identificador de transacción.
-

12. Pruebas

- Aprobar rendición completa.
 - Aprobar con gastos rechazados.
 - Gastos pendientes.
 - Gastos observados.
 - Error documental abierto.
 - Autoaprobación.
 - Rechazo motivado.
 - Rechazo sin motivo.
 - Aprobaciones concurrentes.
 - Cálculos congelados.
 - Permisos.
-

13. Criterios de aceptación

- Solo se aprueban rendiciones revisadas.
 - Todos los gastos tienen decisión.
 - No existe autoaprobación.
 - El rechazo requiere motivo.
 - La aprobación queda firmemente identificada.
 - Los totales se conservan.
 - La operación es transaccional.
-

R013 — Historial de aprobaciones

1. Objetivo

Permitir reconstruir toda la vida de una rendición, incluyendo:

- Presentaciones.
- Revisiones.
- Observaciones.
- Correcciones.
- Aprobaciones.
- Rechazos.
- Reaperturas.
- Liquidación.
- Pagos.
- Conciliaciones.
- Cierre.

El modelo `AprobacionRendicion` ya conserva una secuencia de acciones globales, usuario, comentario y fecha.

2. Fuentes del historial

La línea de tiempo consolidará:

```
AprobacionRendicion
RevisionDetalleRendicion
EventoAuditoria
LiquidacionRendicion
PagoLiquidacionRendicion
AplicacionMovimientoBancario
```

No se duplicarán físicamente todos los datos en una nueva tabla.

Se construirá un selector que unifique los eventos.

3. Selector

```
selectors/historial_rendiciones.py
```

Funciones:


```
obtener_historial_rendicion()
obtener_historial_detalle()
obtener_historial_financiero()
```

Cada evento se normalizará:

```
{
  "fecha": datetime,
  "tipo": "APROBACION",
  "titulo": "Rendición aprobada",
  "detalle": "...",
  "usuario": usuario,
  "objeto": referencia,
  "datos": {},
}
```

4. Línea de tiempo

Ejemplo:

```
10-08-2026 – Creada por Juan
12-08-2026 – Presentada por Juan
13-08-2026 – Revisión iniciada por María
13-08-2026 – Gasto #5 observado
14-08-2026 – Gasto #5 corregido
14-08-2026 – Rendición presentada nuevamente
15-08-2026 – Rendición aprobada
16-08-2026 – Liquidación creada
18-08-2026 – Reembolso ejecutado
19-08-2026 – Reembolso conciliado
19-08-2026 – Rendición cerrada
```

5. Filtros

- Fecha.
 - Tipo de evento.
 - Usuario.
 - Gasto.
 - Acción.
 - Evento financiero.
 - Estado.
-

6. Permisos

Responsable

Puede ver el historial de sus rendiciones.

Revisor

Puede ver el historial de las rendiciones revisadas.

Finanzas

Puede ver liquidaciones y conciliaciones.

Auditor

Puede ver todos los eventos.

Los datos técnicos sensibles, como IP o JSON completo, pueden limitarse a auditor y administrador.

7. URLs

```
/administracion/rendiciones/<pk>/historial/  
/administracion/rendiciones/gastos/<pk>/historial/  
/administracion/rendiciones/<pk>/historial/exportar/
```

8. Vista

```
RendicionHistorialView  
DetalleRendicionHistorialView
```

9. Template

```
templates/administracion/rendiciones/  
├─ historial.html  
├─ historial_detalle.html  
└─ componentes/  
    └─ timeline.html
```

10. Reglas

1. Los eventos no se editan.
 2. Los eventos no se eliminan desde la interfaz.
 3. Deben ordenarse por fecha e identificador.
 4. Las fechas se muestran con zona horaria.
 5. Los valores monetarios deben mostrar CLP.
 6. Los eventos financieros deben identificar montos.
 7. Las observaciones deben vincular el gasto.
 8. La reapertura debe ser claramente visible.
 9. La exportación debe mantener filtros.
 10. Los permisos se verifican por rendición.
-

11. Pruebas

- Historial vacío.
 - Presentación.
 - Observación de gasto.
 - Corrección.
 - Aprobación.
 - Rechazo.
 - Liquidación.
 - Pago parcial.
 - Conciliación.
 - Eventos con misma fecha.
 - Usuario sin permiso.
 - Exportación.
-

12. Criterios de aceptación

- Toda la secuencia está visible.
 - Se identifica quién hizo cada acción.
 - Las observaciones conservan versiones.
 - Los eventos financieros aparecen.
 - El historial no es editable.
 - Los permisos restringen la información.
 - La línea de tiempo puede exportarse.
-

R014 — Liquidación de rendición

1. Objetivo

Congelar el resultado financiero de una rendición aprobada y determinar si:

- Está cuadrada.

- El responsable debe devolver.
- La empresa debe reembolsar.

La Carta Gantt define expresamente esos tres resultados.

2. Modelo principal

LiquidacionRendicion

El modelo ya conserva:

- Total de fondos.
 - Total aprobado.
 - Diferencia.
 - Resultado.
 - Usuario que liquida.
 - Observaciones.
-

3. Fórmula

```
diferencia
=
total fondos entregados
- total gastos aprobados
```

Diferencia cero

CUADRADA

Diferencia positiva

RESPONSABLE_DEVUELVE

Diferencia negativa

EMPRESA_REEMBOLSA

El modelo actual ya valida esta correspondencia.

4. Condiciones previas

Solo puede liquidarse cuando:

1. La rendición está **APROBADA**.
 2. Existe fecha de aprobación.
 3. Todos los gastos tienen decisión.
 4. No existen errores abiertos.
 5. No existe una liquidación previa activa.
 6. Los fondos están definidos.
 7. Los totales pueden recalcularse.
 8. El usuario tiene permiso.
 9. No existe una transacción concurrente.
-

5. Snapshot

Al crear la liquidación se deben congelar:

```
Total de fondos
Total declarado
Total aprobado
Total rechazado
Diferencia
Responsable
Período
Fecha de aprobación
Aprobador
```

Se recomienda agregar:

```
total_gastos_declarados = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

total_gastos_rechazados = models.DecimalField(
    max_digits=14,
    decimal_places=2,
)

datos_snapshot = models.JSONField(
    default=dict,
    blank=True,
)
```

6. Estado inicial

Cuadrada

```
estado = REGULARIZADA
```

No necesita devolución ni reembolso.

Puede quedar pendiente únicamente el cierre administrativo.

Con diferencia

```
estado = PENDIENTE
```

Debe regularizarse antes del cierre.

7. Servicio

```
services/rendiciones/liquidacion.py
```

```
@transaction.atomic
def liquidar_rendicion(
    *,
    rendicion_id,
    usuario,
    observaciones="",
):
    rendicion = (
        Rendicion.objects
        .select_for_update()
        .get(pk=rendicion_id)
    )

    # Verificar APROBADA.
    # Bloquear fondos y detalles.
    # Recalcular totales.
    # Determinar resultado.
    # Crear snapshot.
    # Crear LiquidacionRendicion.
    # Cambiar rendición a LIQUIDADA.
    # Registrar historial.
```

8. URLs

```
/administracion/rendiciones/<pk>/liquidar/  
/administracion/rendiciones/liquidaciones/<pk>/
```

9. Vistas

```
RendicionLiquidarView  
LiquidacionRendicionDetailView
```

10. Pantalla de confirmación

Concepto	Monto
Fondos entregados	\$500.000 CLP
Gastos declarados	\$530.000 CLP
Gastos rechazados	\$20.000 CLP
Gastos aprobados	\$510.000 CLP
Diferencia	-\$10.000 CLP
Resultado	Empresa reembolsa

El usuario no debe ingresar manualmente el resultado.

11. Anulación

Una liquidación no se elimina.

Para anularla:

- Se requiere reapertura.
 - No deben existir pagos conciliados, o deben reversarse.
 - Se exige motivo.
 - Se conserva el snapshot anterior.
 - Se crea un evento de reapertura.
-

12. Permisos

```
liquidar_rendicion  
anular_liquidacionrendicion  
ver_liquidacionrendicion
```

13. Pruebas

- Liquidación cuadrada.
- Responsable devuelve.
- Empresa reembolsa.
- Rendición no aprobada.
- Liquidación duplicada.
- Fondos anulados.
- Gastos aprobados parcialmente.
- Snapshot.
- Concurrencia.
- Anulación controlada.
- Cambio a LIQUIDADA.

14. Criterios de aceptación

- Solo se liquida una rendición aprobada.
- Los totales se recalculan.
- El resultado es automático.
- Los datos quedan congelados.
- No existen dos liquidaciones activas.
- La rendición cambia a LIQUIDADA.
- La operación queda auditada.

R015 — Devolución del responsable

1. Objetivo

Registrar la devolución del dinero que el responsable debe reintegrar cuando los fondos entregados superan los gastos aprobados.

Ejemplo:

Fondos entregados:	\$500.000 CLP
Gastos aprobados:	\$450.000 CLP
Diferencia:	\$50.000 CLP

Resultado:

Responsable debe devolver \$50.000 CLP

2. Modelo utilizado

PagoLiquidacionRendicion

Con:

tipo = DEVOLUCION_RESPONSABLE

El modelo actual permite pagos parciales y evita superar el valor absoluto de la diferencia.

3. Formas de devolución

- Transferencia bancaria.
- Depósito.
- Efectivo.
- Descuento autorizado, si existe política.
- Otro medio documentado.

Se recomienda agregar:

DEPOSITO

a las formas de pago.

4. Devoluciones parciales

Debe permitirse:

Diferencia:	\$100.000 CLP
Devolución 1:	\$60.000 CLP
Devolución 2:	\$40.000 CLP

Estados:

PENDIENTE
→ PARCIAL
→ REGULARIZADA

5. Reglas

1. La liquidación debe indicar RESPONSABLE_DEVUELVE .
2. El pago debe ser de tipo devolución.
3. El monto debe ser positivo.
4. No puede superar el saldo pendiente.
5. El responsable debe coincidir con la rendición.
6. Una devolución ejecutada no se edita libremente.
7. La anulación exige motivo.
8. Una transferencia requiere referencia.
9. Una devolución bancaria requiere movimiento de ingreso.
10. Una devolución en efectivo requiere comprobante.
11. La rendición no se cierra hasta regularizar.
12. Si requiere banco, tampoco se cierra hasta conciliar.

6. Registro versus confirmación

Registro informado por responsable

Estado PENDIENTE

El responsable informa:

- Fecha.
- Monto.
- Medio.
- Referencia.
- Comprobante.

Confirmación por Finanzas

Estado EJECUTADO

Finanzas confirma que el dinero fue recibido.

Conciliación

Estado CONCILIADO

Se vincula con un movimiento bancario de ingreso.

7. URLs

```
/administracion/rendiciones/liquidaciones/<pk>/devoluciones/  
/administracion/rendiciones/liquidaciones/<pk>/devoluciones/nueva/  
/administracion/rendiciones/devoluciones/<pk>/confirmar/  
/administracion/rendiciones/devoluciones/<pk>/anular/
```

8. Vistas

```
DevolucionResponsableListView  
DevolucionResponsableCreateView  
DevolucionResponsableConfirmarView  
DevolucionResponsableAnularView
```

9. Servicio

```
services/rendiciones/devoluciones.py
```

Funciones:

```
registrar_devolucion()  
confirmar_devolucion()  
anular_devolucion()  
actualizar_estado_liquidacion()
```

10. Comprobante

La devolución podrá adjuntar:

- Comprobante de transferencia.
- Comprobante de depósito.
- Recibo de caja.
- Documento de descuento autorizado.

Se reutilizará `AdjuntoAdministracion`.

11. Conciliación bancaria

Para una devolución por transferencia:

```
MovimientoBancario.INGRESO  
→ AplicacionMovimientoBancario  
→ PagoLiquidacionRendicion
```

La validación del puente bancario debe exigir ingreso para `DEVOLUCION_RESPONSABLE`.

12. Auditoría

Registrar:

- Liquidación.
 - Monto.
 - Forma.
 - Referencia.
 - Estado.
 - Usuario que registra.
 - Usuario que confirma.
 - Movimiento bancario.
 - Anulación.
-

13. Pruebas

- Devolución completa.
 - Devolución parcial.
 - Exceso.
 - Tipo incorrecto.
 - Liquidación cuadrada.
 - Liquidación de reembolso.
 - Transferencia sin referencia.
 - Confirmación.
 - Conciliación con ingreso.
 - Rechazar egreso.
 - Anulación.
 - Actualización de saldo.
-

14. Criterios de aceptación

- Se pueden registrar devoluciones.
- Se permiten pagos parciales.
- No se supera la diferencia.
- Se distingue información de confirmación.

- Las transferencias se concilian con ingresos.
- Los comprobantes quedan protegidos.
- La liquidación actualiza su saldo.

R016 — Reembolso al responsable

1. Objetivo

Gestionar el pago que la empresa debe realizar al responsable cuando los gastos aprobados superan los fondos entregados.

Ejemplo:

Fondos entregados:	\$300.000 CLP
Gastos aprobados:	\$380.000 CLP
Diferencia:	-\$80.000 CLP

Resultado:

Empresa debe reembolsar \$80.000 CLP

2. Modelo utilizado

PagoLiquidacionRendicion

Con:

tipo = REEMBOLSO_EMPRESA

3. Orden de pago

La Gantt exige expresamente una orden de pago.

Se recomienda crear:

```
class OrdenReembolsoRendicion(ModeloAuditoria):  
    class Estado(models.TextChoices):  
        BORRADOR = "BORRADOR", "Borrador"  
        SOLICITADA = "SOLICITADA", "Solicitada"
```

```

    APROBADA = "APROBADA", "Aprobada"
    PAGADA = "PAGADA", "Pagada"
    ANULADA = "ANULADA", "Anulada"

    liquidacion = models.OneToOneField(
        LiquidacionRendicion,
        on_delete=models.PROTECT,
        related_name="orden_reembolso",
    )
    responsable = models.ForeignKey(
        ResponsableRendicion,
        on_delete=models.PROTECT,
        related_name="ordenes_reembolso",
    )
    monto = models.DecimalField(
        max_digits=14,
        decimal_places=2,
    )
    estado = models.CharField(
        max_length=20,
        choices=Estado.choices,
        default=Estado.BORRADOR,
    )
    cuenta_destino = models.CharField(
        max_length=100,
        blank=True,
    )
    banco_destino = models.CharField(
        max_length=150,
        blank=True,
    )
    tipo_cuenta = models.CharField(
        max_length=50,
        blank=True,
    )
    observaciones = models.TextField(blank=True)

```

La orden representa la autorización.

`PagoLiquidacionRendicion` representa la ejecución monetaria real.

4. Snapshot bancario del responsable

La orden debe conservar:

- Nombre.
- RUT.
- Banco.
- Tipo de cuenta.

- Número de cuenta.
- Correo.

No debe depender de que esos datos permanezcan iguales en el perfil del responsable.

5. Flujo

```
Liquidación: EMPRESA_REEMBOLSA
→ Crear orden
→ Solicitar aprobación
→ Aprobar orden
→ Ejecutar transferencia
→ Registrar PagoLiquidacionRendicion
→ Conciliar con egreso bancario
→ Cerrar rendición
```

6. Reglas

1. Solo se crea si el resultado es `EMPRESA_REEMBOLSA`.
 2. El monto debe coincidir con el saldo pendiente.
 3. El responsable debe ser el de la rendición.
 4. La orden necesita datos bancarios.
 5. No se ejecuta una orden no aprobada.
 6. No se paga más que el saldo.
 7. Puede haber pagos parciales si la política lo permite.
 8. La transferencia requiere referencia.
 9. El pago debe vincularse con un movimiento de egreso.
 10. Una orden pagada no se modifica.
 11. Anular exige motivo.
 12. No se cierra la rendición hasta completar pago y conciliación.
-

7. URLs

```
/administracion/rendiciones/liquidaciones/<pk>/orden-reembolso/
/administracion/rendiciones/ordenes-reembolso/<pk>/
/administracion/rendiciones/ordenes-reembolso/<pk>/solicitar/
/administracion/rendiciones/ordenes-reembolso/<pk>/aprobar/
/administracion/rendiciones/ordenes-reembolso/<pk>/pagar/
/administracion/rendiciones/ordenes-reembolso/<pk>/anular/
```

8. Vistas

```
OrdenReembolsoCreateView
OrdenReembolsoDetailView
OrdenReembolsoSolicitarView
OrdenReembolsoAprobarView
OrdenReembolsoPagarView
OrdenReembolsoAnularView
```

9. Servicio

```
services/rendiciones/reembolsos.py
```

Funciones:

```
crear_orden_reembolso()
solicitar_orden_reembolso()
aprobar_orden_reembolso()
ejecutar_reembolso()
anular_orden_reembolso()
actualizar_estado_liquidacion()
```

10. Permisos

```
crear_orden_reembolso
aprobar_orden_reembolso
ejecutar_reembolso
anular_orden_reembolso
```

Quien solicita la orden no debería aprobarla cuando se aplique segregación de funciones.

11. Conciliación bancaria

El pago ejecutado se relaciona así:

```
MovimientoBancario.EGRESO
→ AplicacionMovimientoBancario
→ PagoLiquidacionRendicion
```

La validación debe exigir egreso para `REEMBOLSO_EMPRESA`.

12. Auditoría

Registrar:

- Orden.
- Datos bancarios snapshot.
- Monto.
- Solicitante.
- Aprobador.
- Ejecutor.
- Fecha.
- Referencia bancaria.
- Movimiento conciliado.
- Anulación.

13. Pruebas

- Crear orden válida.
- Resultado incorrecto.
- Monto superior.
- Datos bancarios faltantes.
- Solicitar.
- Aprobar.
- Autoaprobación no permitida.
- Ejecutar pago.
- Pago parcial.
- Conciliar con egreso.
- Rechazar ingreso.
- Anular.
- Actualizar liquidación.

14. Criterios de aceptación

- Existe una orden de pago.
 - La orden conserva datos históricos.
 - Se separa aprobación de ejecución.
 - El pago no supera la diferencia.
 - El reembolso se concilia con egreso.
 - La actividad queda auditada.
 - La rendición no se cierra anticipadamente.
-

R017 — Conciliación de liquidaciones

1. Objetivo

Vincular las devoluciones y reembolsos con los movimientos bancarios que demuestran su ejecución efectiva.

2. Modelo puente

Se reutiliza:

`AplicacionMovimientoBancario`

Con destino:

`PAGO_LIQUIDACION`

El modelo ya incluye una FK hacia `PagoLiquidacionRendicion`.

3. Dirección bancaria

Devolución del responsable

`DEVOLUCION_RESPONSABLE`
→ Movimiento INGRESO

Reembolso de empresa

`REEMBOLSO_EMPRESA`
→ Movimiento EGRESO

Esta validación debe agregarse explícitamente al método `clean()` y al servicio bancario.

4. Conciliación parcial

Debe permitirse:

Pago de liquidación: \$100.000 CLP

Movimiento A:	\$60.000 CLP
Movimiento B:	\$40.000 CLP

También:

Movimiento bancario:	\$200.000 CLP
└─ Reembolso A	\$80.000 CLP
└─ Reembolso B	\$70.000 CLP
└─ Otro destino	\$50.000 CLP

5. Propiedades recomendadas

En `PagoLiquidacionRendicion`:

```
@property
def monto_conciliado(self):
    total = self.aplicaciones_bancarias.filter(
        activa=True
    ).aggregate(
        total=Sum("monto")
    )["total"]

    return total or Decimal("0.00")
```

```
@property
def saldo_por_conciliar(self):
    return self.monto - self.monto_conciliado
```

6. Actualización de estados

Pago de liquidación

Conciliado	Estado
0	EJECUTADO
Parcial	PARCIALMENTE_CONCILIADO
Total	CONCILIADO

Liquidación

Situación	Estado
Sin pagos	PENDIENTE
Pago parcial	PARCIAL
Diferencia pagada	REGULARIZADA
Pagos conciliados	CONCILIADA

Rendición

LIQUIDADA
→ CERRADA

solo cuando la liquidación esté completamente regularizada y conciliada cuando corresponda.

7. Efectivo y medios no bancarios

Una devolución en efectivo no siempre produce inmediatamente un movimiento bancario.

Se recomienda usar:

requiere_conciliacion_bancaria

Si es **True**

No puede cerrarse sin aplicación bancaria.

Si es **False**

Debe existir:

- Comprobante.
- Confirmación de Finanzas.
- Motivo del tratamiento no bancario.
- Evento de auditoría.

Esto evita fabricar movimientos bancarios inexistentes.

8. Sugerencia de movimientos candidatos

Se buscarán candidatos por:

- Dirección.

- Monto.
- Fecha.
- Referencia.
- Cuenta bancaria.
- Responsable.
- Saldo disponible.
- Descripción.

La sugerencia no debe conciliar automáticamente en esta tarea.

9. Interfaz

URL:

```
/administracion/rendiciones/liquidaciones/<pk>/conciliar/
```

Mostrar:

```
Liquidación REN-2026-000123
Resultado: Empresa reembolsa
Monto: $80.000 CLP
Pagado: $80.000 CLP
Conciliado: $0 CLP
Pendiente: $80.000 CLP
```

Tabla:

Movimiento	Fecha	Tipo	Monto disponible	Referencia	Aplicar
------------	-------	------	------------------	------------	---------

10. Servicio

```
services/rendiciones/conciliacion.py
```

```
@transaction.atomic
def conciliar_pago_liquidacion(
    *,
    pago_id,
    aplicaciones,
    usuario,
):
    pago = (
        PagoLiquidacionRendicion.objects
        .select_for_update()
```

```

        .select_related("liquidacion")
        .get(pk=pago_id)
    )

    # Bloquear movimientos.
    # Validar dirección.
    # Validar saldos.
    # Crear aplicaciones.
    # Recalcular pago.
    # Recalcular liquidación.
    # Cerrar rendición si corresponde.
    # Registrar auditoría.

```

Funciones:

```

buscar_movimientos_candidatos()
conciliar_pago_liquidacion()
anular_conciliacion_liquidacion()
actualizar_estado_pago()
actualizar_estado_liquidacion()
cerrar_rendicion_si_corresponde()

```

11. Condiciones para cerrar

Una rendición podrá pasar a **CERRADA** cuando:

1. Está **LIQUIDADA**.
2. Existe liquidación activa.
3. El saldo por regularizar es cero.
4. Los pagos no están anulados.
5. Los pagos que requieren banco están conciliados.
6. No existen alertas financieras abiertas.
7. No existen errores de integridad.
8. El usuario tiene permiso de cierre.

Una liquidación cuadrada podrá cerrarse sin pagos.

12. URLs

```

/administracion/rendiciones/liquidaciones/<pk>/conciliar/
/administracion/rendiciones/pagos-liquidacion/<pk>/conciliar/
/administracion/rendiciones/pagos-liquidacion/<pk>/conciliaciones/
/administracion/rendiciones/aplicaciones-bancarias/<pk>/anular/
/administracion/rendiciones/<pk>/cerrar/

```

13. Vistas

```
LiquidacionConciliarView  
PagoLiquidacionConciliarView  
PagoLiquidacionConciliacionesView  
AplicacionLiquidacionAnularView  
RendicionCerrarView
```

14. Permisos

```
conciliar_liquidacionrendicion  
anular_conciliacion_liquidacion  
cerrar_rendicion
```

15. Auditoría

Registrar:

- Pago.
- Liquidación.
- Movimiento.
- Dirección.
- Monto.
- Estado anterior.
- Estado nuevo.
- Usuario.
- Fecha.
- Motivo de reversa.
- Cierre resultante.

16. Pruebas

- Devolución con ingreso.
- Reembolso con egreso.
- Dirección incorrecta.
- Conciliación parcial.
- Conciliación con varios movimientos.
- Movimiento compartido.
- Exceso sobre pago.
- Exceso sobre movimiento.
- Pago anulado.
- Movimiento excluido.
- Operaciones concurrentes.

- Regularización completa.
- Cierre automático controlado.
- Liquidación cuadrada.
- Medio sin conciliación bancaria.

17. Criterios de aceptación

- Las devoluciones se concilian con ingresos.
- Los reembolsos se concilian con egresos.
- Se permiten aplicaciones parciales.
- No se superan pagos ni movimientos.
- Los estados se actualizan.
- Los medios no bancarios se justifican.
- La rendición solo se cierra cuando corresponde.
- Las reversas conservan historial.

7. Estructura de archivos del bloque

```
administracion/
├── models/
│   ├── rendiciones.py
│   └── auditoria.py
├── forms/
│   ├── presentacion_rendicion.py
│   ├── revision_rendicion.py
│   ├── aprobacion_rendicion.py
│   ├── liquidacion_rendicion.py
│   ├── devolucion_rendicion.py
│   ├── reembolso_rendicion.py
│   └── conciliacion_liquidacion.py
├── views/
│   ├── presentacion_rendicion.py
│   ├── revision_rendicion.py
│   ├── aprobacion_rendicion.py
│   ├── historial_rendicion.py
│   ├── liquidacion_rendicion.py
│   ├── devoluciones_rendicion.py
│   ├── reembolsos_rendicion.py
│   └── conciliacion_liquidacion.py
├── services/rendiciones/
│   ├── workflow.py
│   ├── revision.py
│   └── aprobacion.py
```



```
|   ├── historial.py
|   ├── liquidacion.py
|   ├── devoluciones.py
|   ├── reembolsos.py
|   └── conciliacion.py
|
|   ├── selectors/
|   │   ├── rendiciones_revision.py
|   │   ├── historial_rendiciones.py
|   │   ├── liquidaciones.py
|   │   └── movimientos_liquidacion.py
|   |
|   ├── templates/administracion/rendiciones/
|   │   ├── presentar.html
|   │   ├── revisar.html
|   │   ├── aprobar.html
|   │   ├── rechazar.html
|   │   ├── historial.html
|   │   ├── liquidacion/
|   │   ├── devoluciones/
|   │   ├── reembolsos/
|   │   └── conciliacion/
|   |
|   ├── static/administracion/js/
|   │   ├── revision_rendicion.js
|   │   └── conciliacion_liquidacion.js
|   |
|   └── tests/rendiciones/
|       ├── test_presentacion.py
|       ├── test_revision.py
|       ├── test_aprobacion.py
|       ├── test_historial.py
|       ├── test_liquidacion.py
|       ├── test_devoluciones.py
|       ├── test_reembolsos.py
|       ├── test_conciliacion.py
|       └── test_cierre.py
```

8. Migraciones previstas

Migración 1 — Revisión detallada

Crear:

RevisionDetalleRendicion

Agregar índices por:

- Detalle.
- Acción.
- Fecha.
- Usuario.

Migración 2 — Historial general

Agregar a `AprobacionRendicion` :

```
estado anterior
estado nuevo
datos JSON
correlation_id
```

Migración 3 — Asignación de revisión

Agregar a `Rendicion` :

```
revisor_actual
revision_iniciada_en
declaración aceptada
```

Migración 4 — Liquidación

Agregar:

```
estado
total declarado
total rechazado
snapshot JSON
motivo de anulación
```

Migración 5 — Pago de liquidación

Agregar:

```
estado
fecha de ejecución
ejecutado por
requiere conciliación bancaria
motivo de anulación
```

Migración 6 — Orden de reembolso

Crear:

```
OrdenReembolsoRendicion
```

Migración 7 — Integridad

Agregar restricciones para:

- Pago de devolución compatible con resultado.
- Pago de reembolso compatible con resultado.
- Montos positivos.
- Estados y fechas consistentes.
- Una orden activa por liquidación.
- Saldo no negativo.

9. Índices recomendados

Rendición

```
models.Index(
    fields=["estado", "revisor_actual"],
)
models.Index(
    fields=["estado", "fecha_presentacion"],
)
models.Index(
    fields=["estado", "fecha_aprobacion"],
)
```

AprobacionRendicion

```
models.Index(  
    fields=["rendicion", "-fecha"],  
)  
models.Index(  
    fields=["accion", "-fecha"],  
)
```

RevisionDetalleRendicion

```
models.Index(  
    fields=["detalle", "-fecha"],  
)  
models.Index(  
    fields=["usuario", "-fecha"],  
)
```

LiquidacionRendicion

```
models.Index(  
    fields=["estado", "resultado"],  
)  
models.Index(  
    fields=["fecha"],  
)
```

PagoLiquidacionRendicion

```
models.Index(  
    fields=["liquidacion", "estado"],  
)  
models.Index(  
    fields=["tipo", "estado"],  
)  
models.Index(  
    fields=["fecha", "estado"],  
)
```

OrdenReembolsoRendicion

```
models.Index(  
    fields=["estado", "-creado_en"],  
)
```

```
models.Index(  
    fields=["responsable", "estado"],  
)
```

10. Definición de terminado del bloque

R010–R017 se considerará terminado cuando:

- Una rendición válida puede presentarse.
- La presentación bloquea la edición.
- El responsable acepta una declaración.
- Un revisor puede tomar la rendición.
- Cada gasto puede aprobarse, observarse o rechazarse.
- Las aprobaciones parciales se justifican.
- Las observaciones pueden corregirse.
- El historial no se sobrescribe.
- La rendición puede aprobarse o rechazarse.
- No existe autoaprobación.
- Se crea una liquidación con snapshot.
- El resultado se calcula automáticamente.
- Se permiten devoluciones parciales.
- Se permite generar una orden de reembolso.
- La orden puede aprobarse y pagarse.
- Las devoluciones se vinculan con ingresos bancarios.
- Los reembolsos se vinculan con egresos bancarios.
- No se sobreaplican pagos ni movimientos.
- Los estados de pago y liquidación se recalculan.
- La rendición se cierra únicamente después de regularizar.
- Todas las operaciones quedan auditadas.
- Las pruebas automáticas pasan.

11. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Presentar datos incompletos	Alto	Validación integral previa
Editar después de presentar	Alto	Bloqueo por estado
Perder observaciones anteriores	Alto	Historial por gasto
Dos revisores modifican a la vez	Alto	Asignación y bloqueo
Autoaprobación	Alto	Permiso y validación por objeto
Aprobar gastos pendientes	Alto	Validación de decisión final
Cambiar totales tras liquidar	Alto	Snapshot

Riesgo	Impacto	Tratamiento
Liquidar dos veces	Alto	One-to-one y transacción
Devolver más que la diferencia	Alto	Control de saldo
Reembolsar más que la diferencia	Alto	Control de orden y pago
Conciliar ingreso como egreso	Alto	Validación por tipo
Cerrar sin recibir devolución	Alto	Estado de regularización
Cerrar sin pagar reembolso	Alto	Control de pago
Pago en efectivo sin respaldo	Alto	Evidencia y confirmación
Reversar sin historial	Alto	Anulación lógica
Concurrencia bancaria	Alto	<code>select_for_update()</code>
Orden de pago sin segregación	Medio/alto	Solicitante y aprobador separados

12. Resultado consolidado

Al finalizar este bloque existirá el siguiente ciclo:

```

Rendición preparada
→ presentación
→ revisión
→ observaciones o decisión
→ aprobación
→ liquidación
├─ devolución del responsable
├─ reembolso de la empresa
→ conciliación bancaria
→ cierre

```

La salida de este bloque habilita:

```

R018 – Alertas de rendiciones
R019 – Exportación PDF y Excel
R020 – Dashboard
R021 – Pruebas del módulo
R022 – Manual de usuario

```